TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

KHOA TOÁN - TIN HỌC



BÁO CÁO THỰC HÀNH

Môn học: Xử lý dữ liệu lớn

Bài thực hành L01

HDFS: tổ chức block, nhân bản và khả năng chịu lỗi DataNode

Sinh viên thực hiện: Nguyễn Thị Huyền Trang
Mã số sinh viên: 23280092
Lớp: 23KDL

TP. Hồ Chí Minh, Tháng 10 năm 2026

Mục lục

1 Mục tiêu và môi trường **2**

2 Thực hiện và kết quả **3**

2.1 Khởi động cluster . . . 3
2.2 Bộ lệnh `hdfs dfs` . . . 5
2.3 Block size mặc định và block 16 MB . . . 6
2.4 Thay đổi hệ số nhân bản . . . 8
2.5 Dừng một DataNode và tái nhân bản . . . 9
2.6 Safe mode và fsimage . . . 14

3 Trả lời câu hỏi checkpoint **15**

3.1 Tệp 300 MB với cấu hình mặc định chiếm bao nhiêu block, replica và dung lượng? 15
3.2 Vì sao block cuối của tệp taxi nhỏ hơn 16 MB, và có tốn 16 MB đĩa không? . . 15
3.3 Vị trí block được lưu ở đâu, và khi NameNode khởi động lại thì sao? . . . 15
3.4 Vì sao tệp hệ số 2 được phục hồi còn tệp hệ số 3 thì không? . . . 15
3.5 `fsck` báo gì khi có block under-replicated, và vì sao đó chưa phải lỗi? . . . 15
3.6 Nếu giữ `recheck-interval` mặc định 5 phút thì sao, và vì sao timeout dài lại tốt? . . . 16
3.7 Vì sao NameNode khởi động trong safe mode và phải chặn ghi? . . . 16
3.8 Vì sao 1 triệu tệp 1 KB là vấn đề, còn 1 tệp 1 KB thì không? . . . 16

4 Kết luận **16**

Tài liệu tham khảo **17**

Phụ lục: Tệp nộp kèm và trích đoạn lệnh **17**

Danh mục hình

1 Trang Overview của NameNode khi cluster khỏe mạnh: 3 Live Nodes, 0 Dead Nodes, 0 block under-replicated. . . . 4
2 Trang Datanodes của NameNode khi cả ba DataNode đều ở trạng thái In service. 5
3 Đầu ra `dfsadmin -report` (3 DataNode sống, ánh xạ IP sang hostname), chuỗi lệnh `hdfs dfs` và kết quả `hdfs dfs -stat`. . . . 6
4 Kết quả `fsck -files -blocks -locations` của tệp Parquet với block mặc định (1 block) và block 16 MB (3 block). . . . 7
5 Trang Browse Directory hiển thị thư mục `/data/taxi` với hệ số nhân bản và block size của từng mục. . . . 7
6 Tệp block vật lý `blk_<id>` và `blk_<id>_<genstamp>.meta` trong thư mục dữ liệu của datanode1. . . . 8

7 Kết quả `setrep -w 2`, `du -h` sau khi giảm hệ số và tệp được ghi với hệ số nhân bản 1. . . . 9
8 `fsck` của tệp block 16 MB trước khi dừng datanode3: mỗi block có 2 replica trên datanode1 và datanode3. . . . 9
9 Dòng thời gian thăm dò `dfsadmin -report` sau khi dừng datanode3 và dòng log `removeDeadDatanode` của NameNode. . . . 10
10 Trang Overview của NameNode khi datanode3 đã bị tuyên bố chết. . . . 11
11 Trang Datanodes khi datanode3 chết: datanode1 và datanode2 In service, datanode3 ở trạng thái Down. . . . 12
12 `fsck` của tệp block 16 MB sau khi dừng datanode3: replica đã chuyển sang datanode2, tệp trở lại trạng thái OK. . . . 12
13 `fsck /data` khi datanode3 chết: hai tệp hệ số 3 under-replicated nhưng HEALTHY; md5 của tệp tải về trùng bản gốc. . . . 13
14 Trạng thái cluster sau khi datanode3 quay lại: không còn block under-replicated hay over-replicated. . . . 13
15 Safe mode thủ công (ghi bị từ chối, đọc vẫn hoạt động), safe mode tự động khi khởi động lại và nội dung fsimage. . . . 14

Danh mục bảng

1 Phần cứng, phần mềm và cấu hình HDFS dùng trong thực nghiệm. . . . 3
2 Ánh xạ địa chỉ IP sang hostname của các DataNode. . . . 3
3 Số block của cùng một tệp khi dùng block 128 MB và 16 MB. . . . 6
4 Các tệp block vật lý trên datanode1 và tệp checksum đi kèm. . . . 8
5 Dung lượng logic và dung lượng thô theo `hdfs dfs -du -h` sau khi `setrep 2`. 8
6 Diễn biến sau khi dừng datanode3 theo `dfsadmin -report` và log NameNode. 10
7 Vị trí replica của tệp `small_blocks` (hệ số 2) trước và sau khi dừng datanode3. 12
8 Trạng thái các tệp trong `/data` khi chỉ còn 2 DataNode sống. . . . 13
9 Danh mục tệp log nộp kèm báo cáo. . . . 17

1 Mục tiêu và môi trường

Bài lab dựng một cluster HDFS nhiều node bằng Docker Compose, dùng bộ lệnh `hdfs dfs`, dùng `hdfs fsck -files -blocks -locations` để xem tệp được chia thành block và replica, thay đổi block size và hệ số nhân bản, dừng một DataNode để quan sát under-replication và tái nhân bản, cuối cùng thử safe mode và đọc fsimage.

Cluster chạy trên một máy (Bảng 1). `recheck-interval` được rút xuống 30 s để thấy sự cố DataNode trong khoảng 90 s thay vì hơn 10 phút. Mọi lệnh chạy trong container `namenode` (đóng vai trò edge node), trừ `docker compose` chạy trên host.

Bảng 1. Phần cứng, phần mềm và cấu hình HDFS dùng trong thực nghiệm.

Thành phần	Giá trị
Máy chủ	WSL2, x86_64, 12 CPU, bộ nhớ Docker 31.9 GB
Phần mềm	Hadoop 3.4.3 (image `de-labs/hadoop:3.4.3`), Docker Compose
Dịch vụ	1 NameNode, 3 DataNode, ResourceManager, NodeManager, JobHistory
Topology	1 rack (`/default-rack`)
`dfs.replication`	3
`dfs.blocksize`	134,217,728 byte (128 MB, mặc định)
`dfs.heartbeat.interval`	3 s
`dfs.namenode.heartbeat.recheck-interval`	30,000 ms (mặc định 300,000 ms)
`dfs.namenode.stale.datanode.interval`	15,000 ms (mặc định 30,000 ms)
`dfs.namenode.redundancy.interval.seconds`	3 s
`dfs.namenode.safemode.extension`	5,000 ms (mặc định 30,000 ms)
`dfs.permissions.enabled`	false (chỉ dùng cho môi trường học tập)

Dữ liệu là hai tệp NYC Taxi đọc từ thư mục `/datasets` bind mount vào container: `taxi_zone_lookup.csv` (12,331 byte) và `yellow_tripdata_2024-01.parquet` (49,961,641 byte, `hdfs dfs -du -h` hiển thị 47.6 M).

2 Thực hiện và kết quả

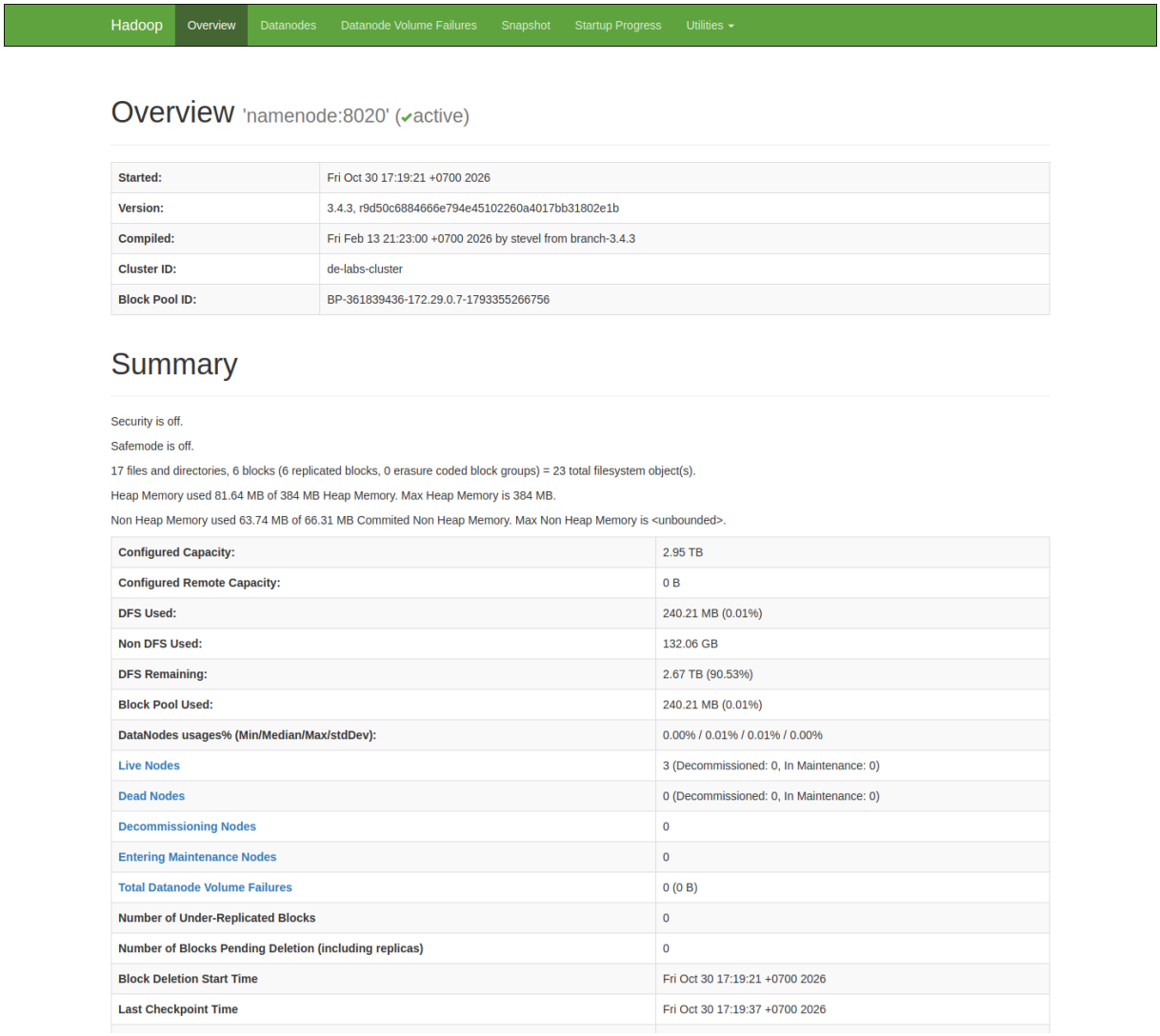
2.1 Khởi động cluster

Sau khi khởi động, `hdfs dfsadmin -report` ghi nhận `Live datanodes (3)`, safe mode OFF. Vì `fsck` in vị trí replica theo IP, chúng tôi ghi lại ánh xạ IP sang hostname ngay lúc này (Bảng 2). Block Pool ID `BP-361839436-172.29.0.7-1793355266756` cho biết NameNode ở 172.29.0.7.

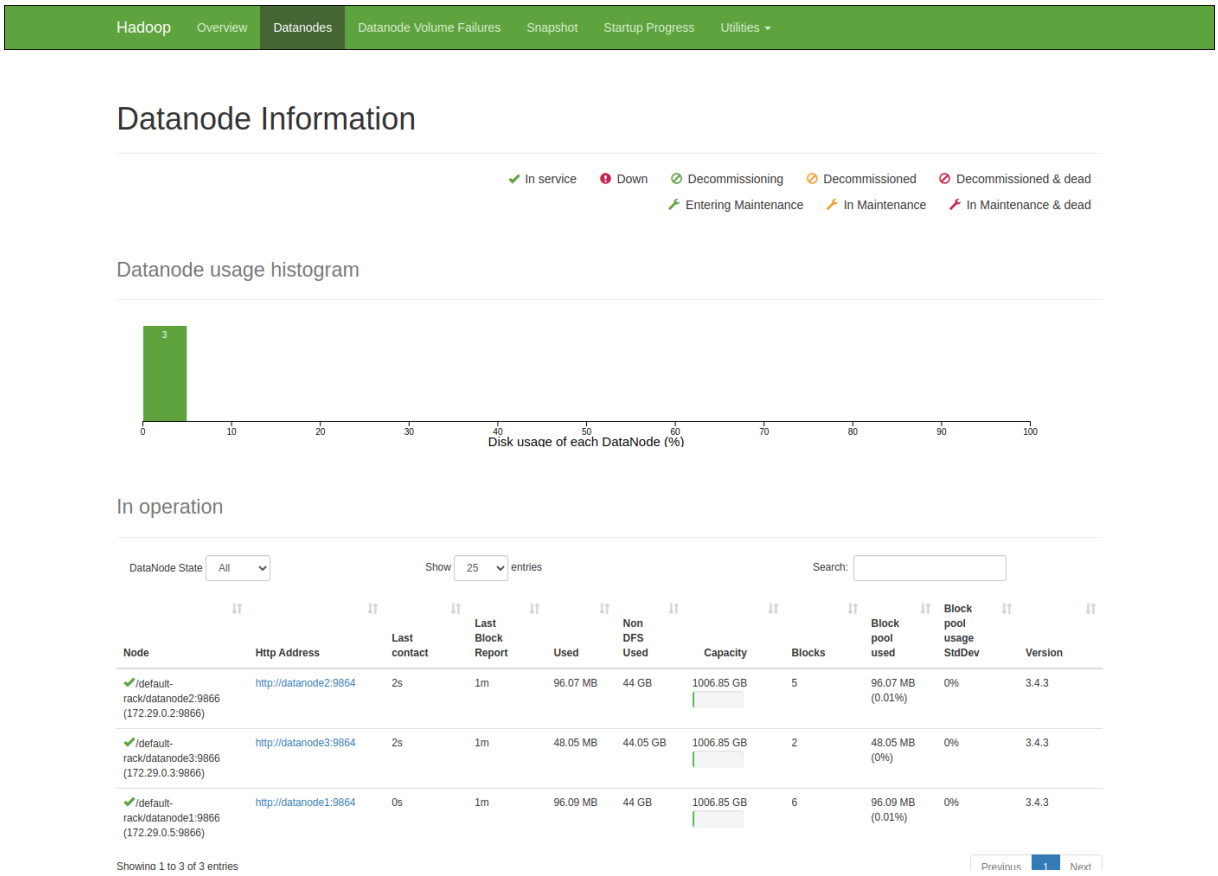
Bảng 2. Ánh xạ địa chỉ IP sang hostname của các DataNode.

Địa chỉ IP	Hostname
172.29.0.5	datanode1
172.29.0.2	datanode2
172.29.0.3	datanode3

Trên giao diện web cổng 9870, trang Overview báo NameNode `active`, phiên bản 3.4.3, safe mode tắt, 3 node sống, 0 node chết, 0 block under-replicated (Hình 1). Trang Datanodes liệt kê cả ba node ở trạng thái In service (Hình 2).



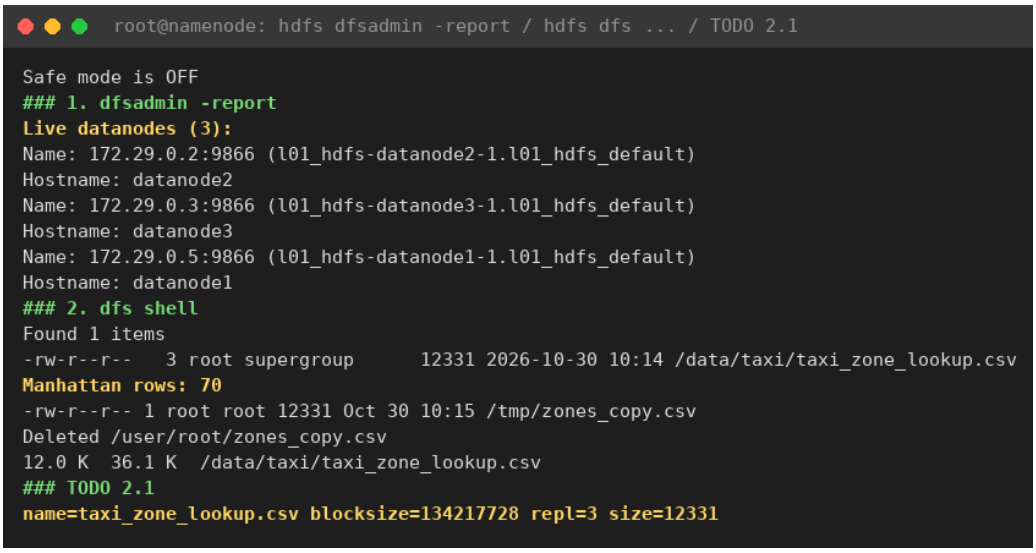
Hình 1. Trang Overview của NameNode khi cluster khỏe mạnh: 3 Live Nodes, 0 Dead Nodes, 0 block under-replicated.



Hình 2. Trang Datanodes của NameNode khi cả ba DataNode đều ở trạng thái In service.

2.2 Bộ lệnh `hdfs dfs`

Các lệnh `hdfs dfs` đều chạy được (Hình 3). `taxi_zone_lookup.csv` có hệ số 3 (cột thứ hai của `ls`), `cat` kết hợp `grep -c` đếm được 70 dòng Manhattan, bản tải về `/tmp/zones_copy.csv` đúng 12,331 byte. `du -h` in 12.0 K dung lượng logic và 36.1 K dung lượng thô, gấp ba do nhân bản. Lệnh `hdfs dfs -stat` (Phụ lục) trả về `blocksize=134217728 repl=3 size=12331`: tệp 12 KB vẫn được gán block size 128 MB.



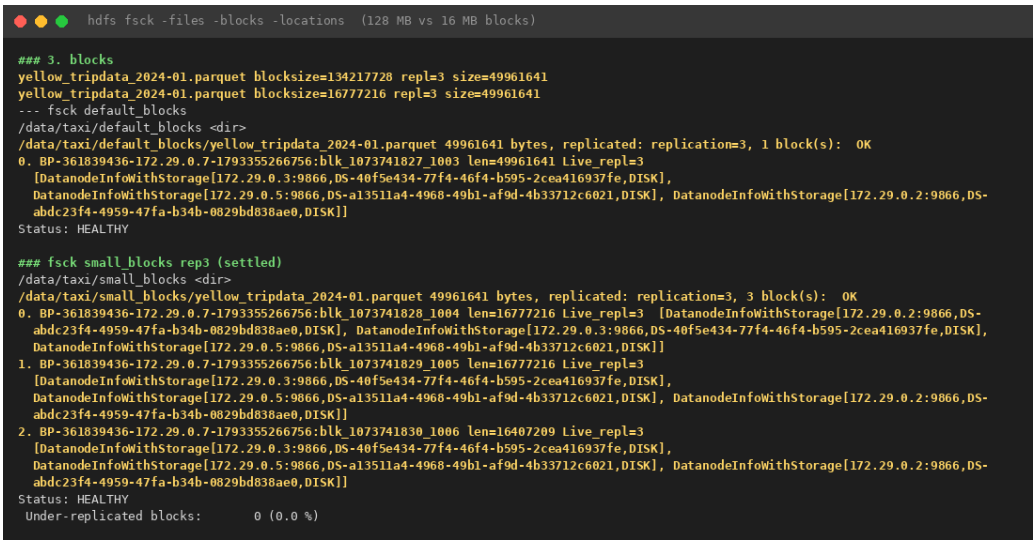
Hình 3. Đầu ra `dfsadmin -report` (3 DataNode sống, ánh xạ IP sang hostname), chuỗi lệnh `hdfs dfs` và kết quả `hdfs dfs -stat`.

2.3 Block size mặc định và block 16 MB

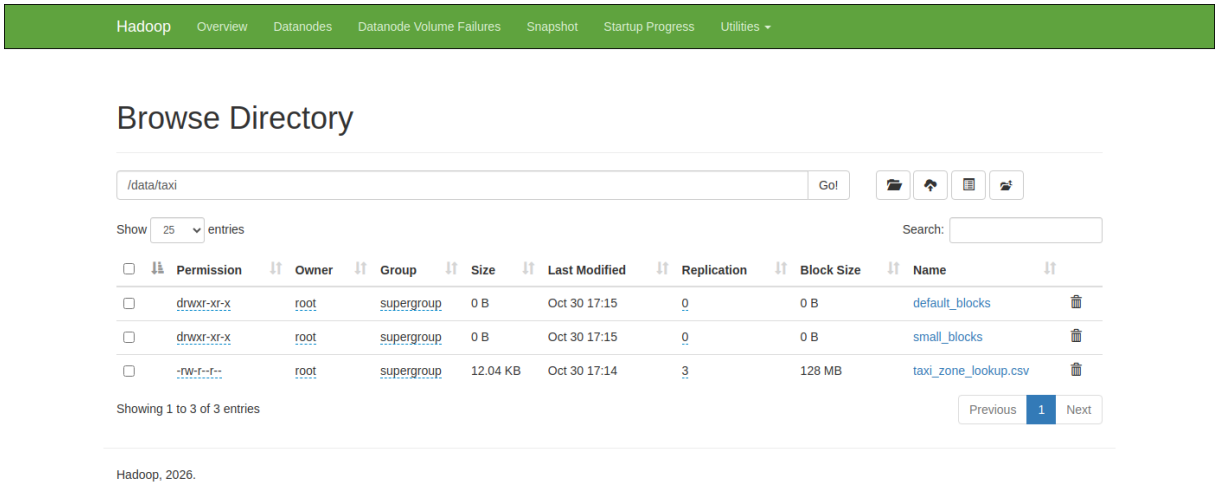
Tệp Parquet được tải lên hai lần: block mặc định vào `/data/taxi/default_blocks` và `-D dfs.blocksize=16m` vào `/data/taxi/small_blocks`. `hdfs dfs -stat` cho bản thứ nhất `blocksize=134217728`, bản thứ hai `blocksize=16777216`, cả hai `repl=3`. Block size chỉ là giới hạn trên, block cuối chỉ lớn bằng phần dữ liệu còn lại [3]. Kết quả `fsck` ở Bảng 3 và Hình 4; trang Browse Directory cũng hiển thị `default_blocks`, `small_blocks` và `taxi_zone_lookup.csv` trong `/data/taxi` với Replication 3, Block Size 128 MB (Hình 5).

Bảng 3. Số block của cùng một tệp khi dùng block 128 MB và 16 MB.

Cách tải lên	Block size (byte)	Số block	Kích thước từng block (byte)
Mặc định	134,217,728	1	49,961,641
`-D dfs.blocksize=16m`	16,777,216	3	16,777,216 + 16,777,216 + 16,407,209



Hình 4. Kết quả `fsck -files -blocks -locations` của tệp Parquet với block mặc định (1 block) và block 16 MB (3 block).



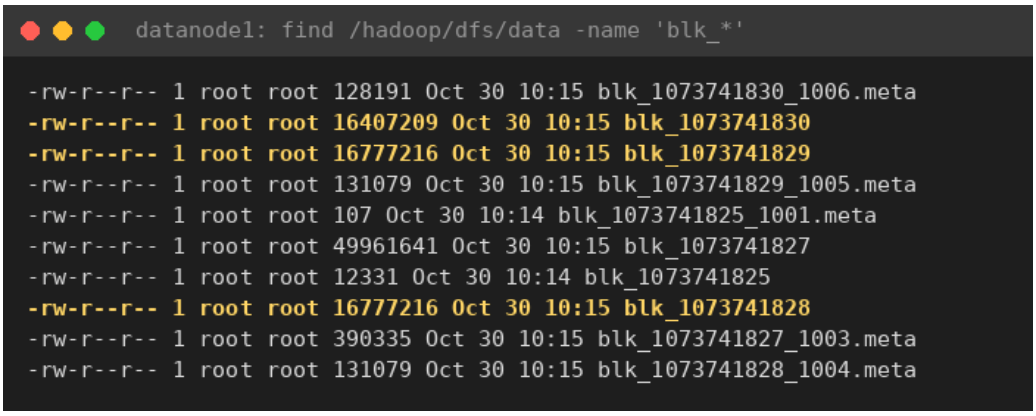
Hình 5. Trang Browse Directory hiển thị thư mục `/data/taxi` với hệ số nhân bản và block size của từng mục.

Ở lần `fsck` đầu tiên trên `small_blocks` (10:15:21, ngay sau khi ghi), hai block `blk_1073741829` và `blk_1073741830` chỉ có `Live_repl=2`, kèm `Under-replicated blocks: 2 (66.666664 %)` và `Missing replicas: 2 (22.222221 %)`, dù trạng thái vẫn `HEALTHY`. Lúc đầu chúng tôi nghĩ việc ghi bị lỗi, nhưng lần `fsck` tiếp theo (đầu tệp `02_setrep.txt`) đã cho cả ba block `Live_repl=3` và `Under-replicated blocks: 0`. NameNode chỉ cập nhật vị trí replica khi DataNode báo về, nên sau khi ghi có một khoảng trễ ngắn.

Trên datanode1, mỗi block là một tệp `blk_<id>` kèm tệp `blk_<id>_<genstamp>.meta` chứa checksum (Hình 6, Bảng 4). Block cuối chỉ có 16,407,209 byte, không bị đệm lên 16 MB. Tệp `.meta` 131,079 byte của block 16,777,216 byte khớp với 4 byte CRC cho mỗi 512 byte dữ liệu ($16{,}777{,}216/512 \times 4 = 131{,}072$) cộng 7 byte header, tức chi phí checksum khoảng 0.78 %.

Bảng 4. Các tệp block vật lý trên datanode1 và tệp checksum đi kèm.

Tệp block	Dữ liệu (byte)	`.meta` (byte)	Thuộc tệp
`blk_1073741825`	12,331	107	`taxi_zone_lookup.csv`
`blk_1073741827`	49,961,641	390,335	`default_blocks/...parquet`
`blk_1073741828`	16,777,216	131,079	`small_blocks/...parquet`, block 0
`blk_1073741829`	16,777,216	131,079	`small_blocks/...parquet`, block 1
`blk_1073741830`	16,407,209	128,191	`small_blocks/...parquet`, block 2



Hình 6. Tệp block vật lý `blk_<id>` và `blk_<id>_<genstamp>.meta` trong thư mục dữ liệu của datanode1.

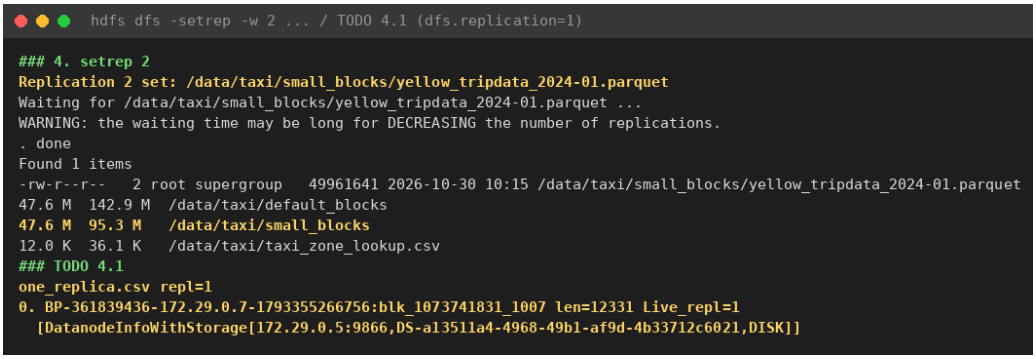
2.4 Thay đổi hệ số nhân bản

`hdfs dfs -setrep -w 2` trên tệp `small_blocks` in cảnh báo rằng việc giảm hệ số có thể phải chờ lâu, nhưng thực tế kết thúc gần như ngay (`. done`). Sau đó `ls` hiển thị hệ số 2 và `fsck` cho `Live_repl=2` ở cả ba block. Dung lượng thô của `small_blocks` giảm đúng một phần ba so với bản `default_blocks` còn hệ số 3 (Bảng 5).

Bảng 5. Dung lượng logic và dung lượng thô theo `hdfs dfs -du -h` sau khi `setrep 2`.

Đường dẫn	Hệ số	Dung lượng logic	Dung lượng thô
`/data/taxi/default_blocks`	3	47.6 M	142.9 M
`/data/taxi/small_blocks`	2	47.6 M	95.3 M
`/data/taxi/taxi_zone_lookup.csv`	3	12.0 K	36.1 K

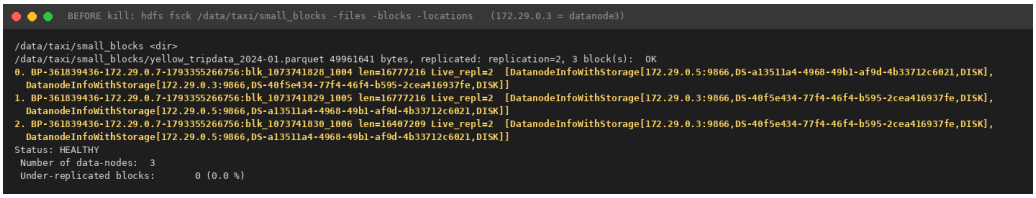
Hệ số nhân bản là tham số phía client, nên để ghi tệp với một replica mà không cần `setrep` chỉ cần ghi đè `-D dfs.replication=1` ngay trong lệnh `put` (Phụ lục). Kết quả là `one_replica.csv repl=1`, block duy nhất `blk_1073741831` (12,331 byte, `Live_repl=1`) nằm trên 172.29.0.5, tức datanode1 (Hình 7).



Hình 7. Kết quả `setrep -w 2`, `du -h` sau khi giảm hệ số và tệp được ghi với hệ số nhân bản 1.

2.5 Dừng một DataNode và tái nhân bản

Ngay trước khi dừng node (10:15:58), cả ba block của `small_blocks` đều có replica trên datanode1 và datanode3, không block nào nằm trên datanode2 (Hình 8, cột "Trước sự cố" của Bảng 7). Vì vậy chúng tôi chọn dừng datanode3: node này giữ replica của mọi block thuộc tệp hệ số 2.



Hình 8. `fsck` của tệp block 16 MB trước khi dừng datanode3: mỗi block có 2 replica trên datanode1 và datanode3.

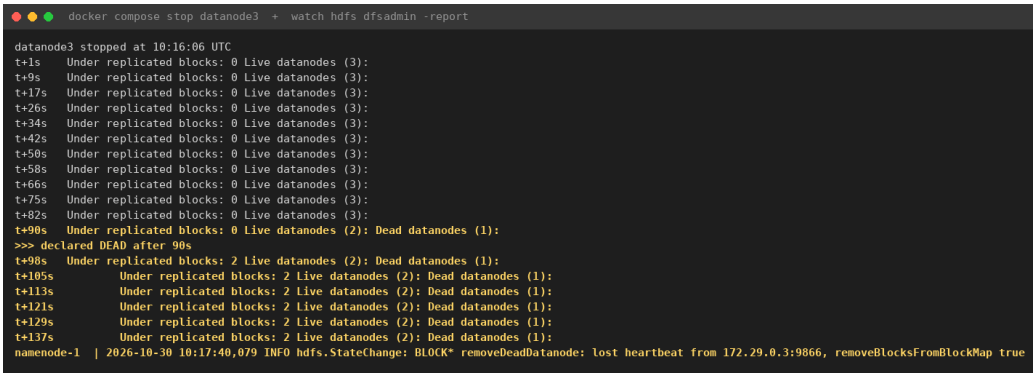
Datanode3 bị dừng bằng `docker compose stop` lúc 10:16:06 UTC, sau đó `hdfs dfsadmin -report` được thăm dò trong vòng lặp `sleep 5` (chu kỳ thực tế khoảng 8 s). DataNode gửi heartbeat mỗi 3 s; node im lặng bị coi là dead sau

$$T_{dead} = 2 \times \text{recheck-interval} + 10 \times \text{heartbeat-interval}$$

[5]. Với cấu hình lab, $T_{dead} = 2 \times 30 + 10 \times 3 = 90$ s. Diễn biến đo được ở Bảng 6 và Hình 9.

Bảng 6. Diễn biến sau khi dừng datanode3 theo `dfsadmin -report` và log NameNode.

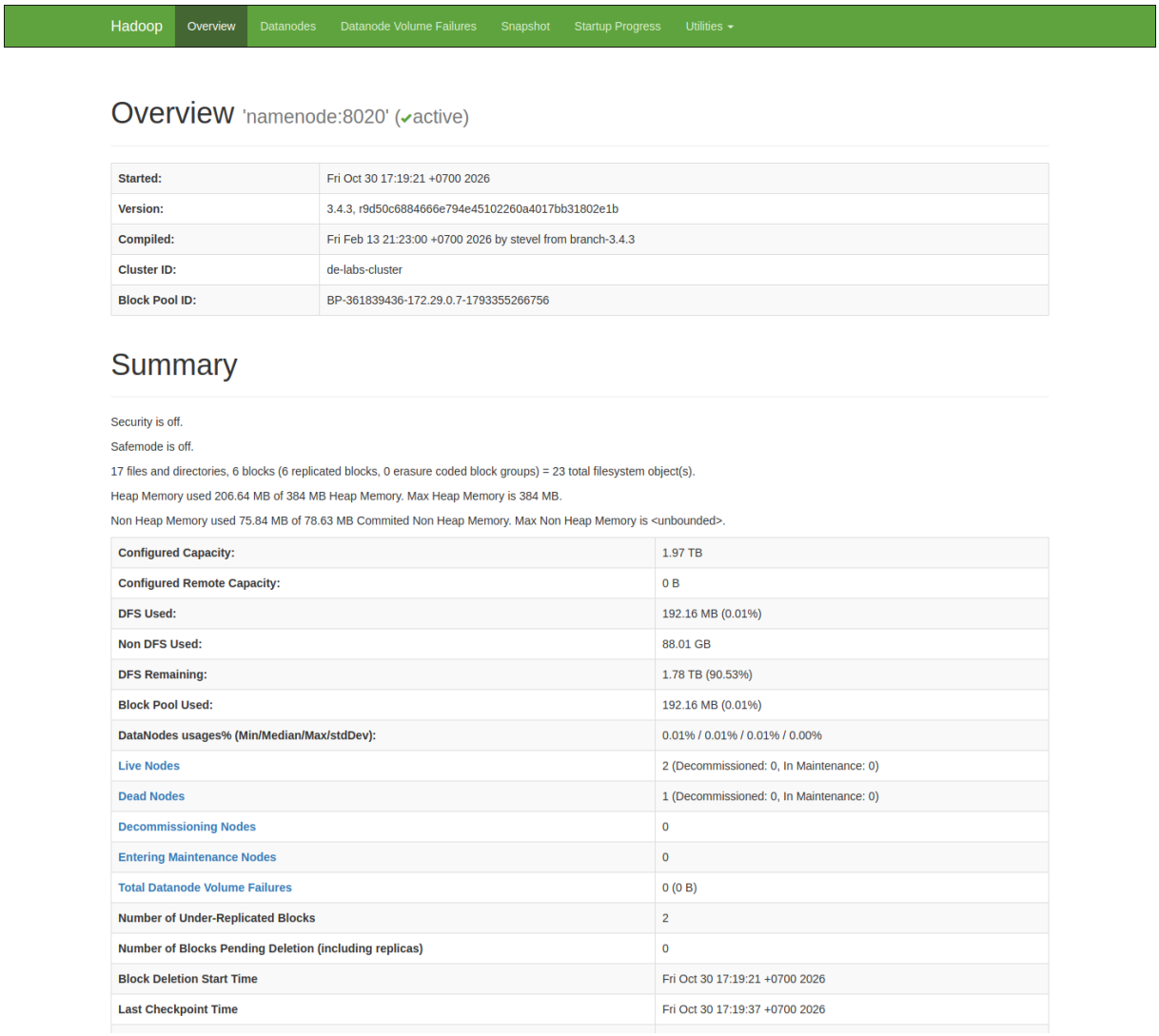
Thời điểm	Sự kiện
t+0 s (10:16:06)	`docker compose stop datanode3`
t+1 s đến t+82 s	`Live datanodes (3)`, `Under replicated blocks: 0` (11 lần thăm dò)
t+90 s	`Live datanodes (2)`, `Dead datanodes (1)`: node bị tuyên bố chết
t+94 s (10:17:40,079)	Log NameNode: `removeDeadDatanode: lost heartbeat from 172.29.0.3:9866`
t+98 s đến t+137 s	`Under replicated blocks: 2` ổn định qua 6 lần thăm dò



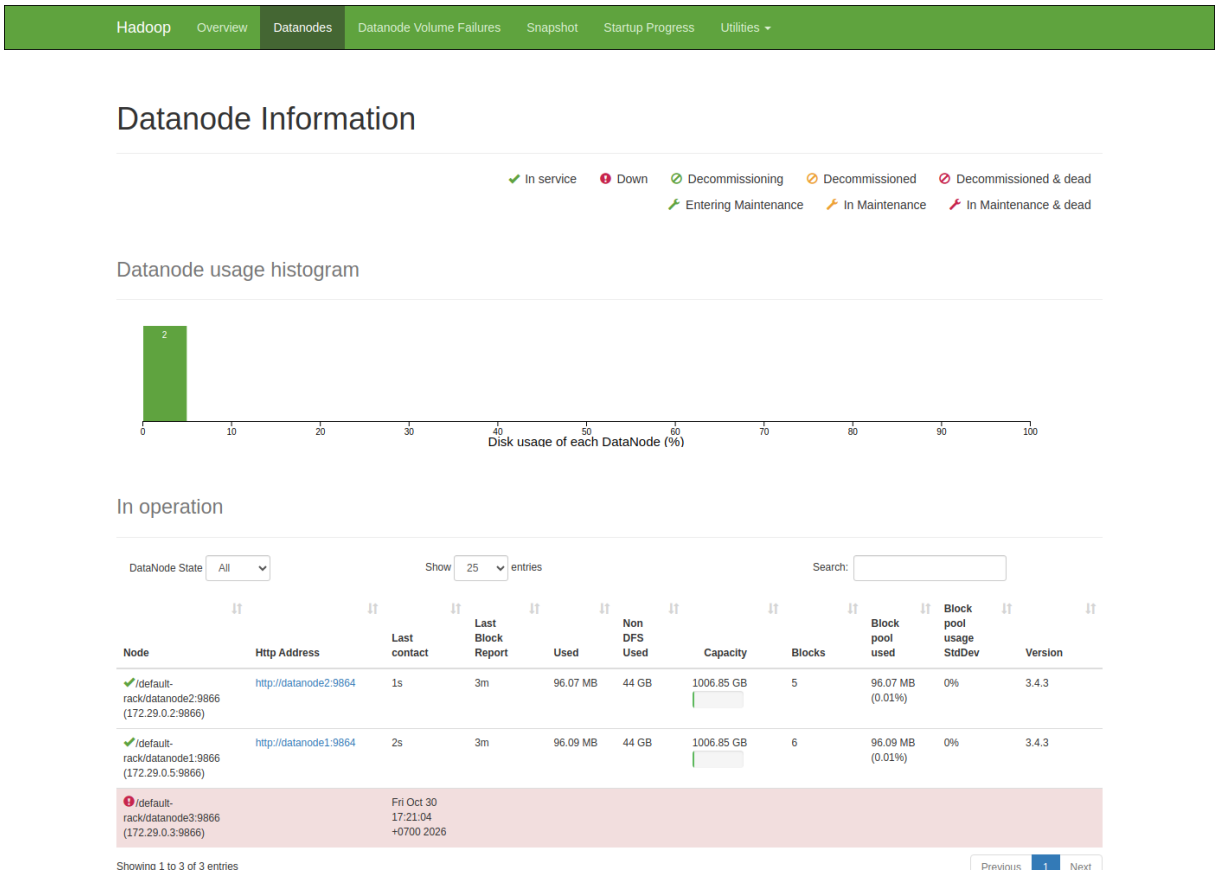
Hình 9. Dòng thời gian thăm dò `dfsadmin -report` sau khi dừng datanode3 và dòng log `removeDeadDatanode` của NameNode.

Node bị tuyên bố chết đúng sau **90 s**, khớp công thức. Replica bị gỡ khỏi block map muộn hơn 4 s vì bộ giám sát heartbeat chỉ chạy theo chu kỳ recheck-interval, và phải sau mốc này chỉ số under-replicated mới tăng. Có một điểm lúc đầu chúng tôi thấy lạ: ba block của tệp hệ số 2 cũng mất một replica, nhưng không lần thăm dò nào ghi nhận quá 2 block under-replicated. Cách giải thích hợp lý là ba block đó đã được tái nhân bản trong khoảng t+94 s đến t+98 s, nhanh hơn chu kỳ thăm dò. Giá trị 2 còn lại là hai block của các tệp hệ số 3 (Bảng 8).

Trên giao diện web, trang Overview hiện một Dead Node (Hình 10) và trang Datanodes đánh dấu datanode3 màu đỏ kèm thời điểm liên lạc cuối (Hình 11). Lần đo chính chúng tôi không kịp chụp giao diện, nên hai ảnh này được chụp ở một lần dừng datanode3 khác; mốc "Last contact" 17:21:04 +0700 (10:21:04 UTC) muộn hơn dòng thời gian trong log và không thuộc lần đo ở Bảng 6.



Hình 10. Trang Overview của NameNode khi datanode3 đã bị tuyên bố chết.

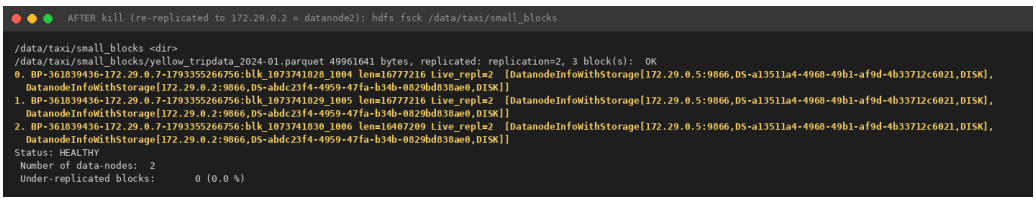


Hình 11. Trang Datanodes khi datanode3 chết: datanode1 và datanode2 In service, datanode3 ở trạng thái Down.

`fsck` lúc 10:18:38 cho thấy `small_blocks` đã trở lại `OK`: `Number of data-nodes: 2`, cả ba block `Live_repl=2`, `Under-replicated blocks: 0` (Hình 12). Replica trên datanode3 đã được chép sang datanode2, node còn sống duy nhất chưa giữ bản sao (Bảng 7).

Bảng 7. Vị trí replica của tệp `small_blocks` (hệ số 2) trước và sau khi dừng datanode3.

Block	Kích thước (byte)	Trước sự cố	Sau sự cố
`blk_1073741828`	16,777,216	datanode1, datanode3	datanode1, datanode2
`blk_1073741829`	16,777,216	datanode3, datanode1	datanode1, datanode2
`blk_1073741830`	16,407,209	datanode3, datanode1	datanode1, datanode2

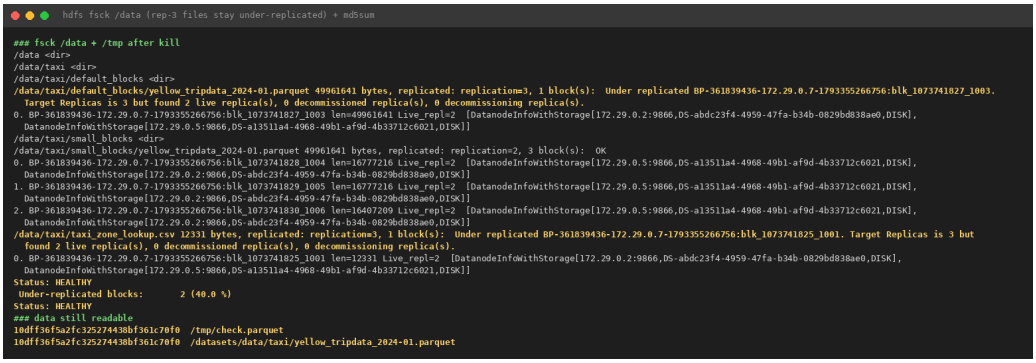


Hình 12. `fsck` của tệp block 16 MB sau khi dừng datanode3: replica đã chuyển sang datanode2, tệp trở lại trạng thái OK.

Với toàn bộ `/data`, `fsck` báo **`Under-replicated blocks: 2 (40.0 %)`** (2 trên 5 block) và `Status: HEALTHY` (Hình 13). Hai block này thuộc hai tệp hệ số 3, mỗi block còn 2 replica trên datanode2 và datanode1 (Bảng 8). Tệp Parquet hệ số 3 tải về có md5 `10dff36f5a2fc325274438bf361c70f0`, trùng với bản gốc. Tệp `/tmp/one_replica.csv` không bị MISSING vì replica duy nhất của nó nằm trên datanode1.

Bảng 8. Trạng thái các tệp trong `/data` khi chỉ còn 2 DataNode sống.

Tệp	Hệ số đích	Block	`Live_repl`	Trạng thái
`default_blocks/...parquet`	3	`blk_1073741827`	2	Under replicated
`small_blocks/...parquet`	2	3 block	2	OK
`taxi_zone_lookup.csv`	3	`blk_1073741825`	2	Under replicated



Hình 13. `fsck /data` khi datanode3 chết: hai tệp hệ số 3 under-replicated nhưng HEALTHY; md5 của tệp tải về trùng bản gốc.

Khởi động lại datanode3, `dfsadmin -report` trở về `Live datanodes (3)` và `fsck` báo `Under-replicated blocks: 0`, `Over-replicated blocks: 0`, `Status: HEALTHY` (Hình 14). Các block hệ số 3 đã có lại replica thứ ba; tệp hệ số 2 lúc này có replica trên cả ba node, và NameNode đã xóa bớt bản thừa.

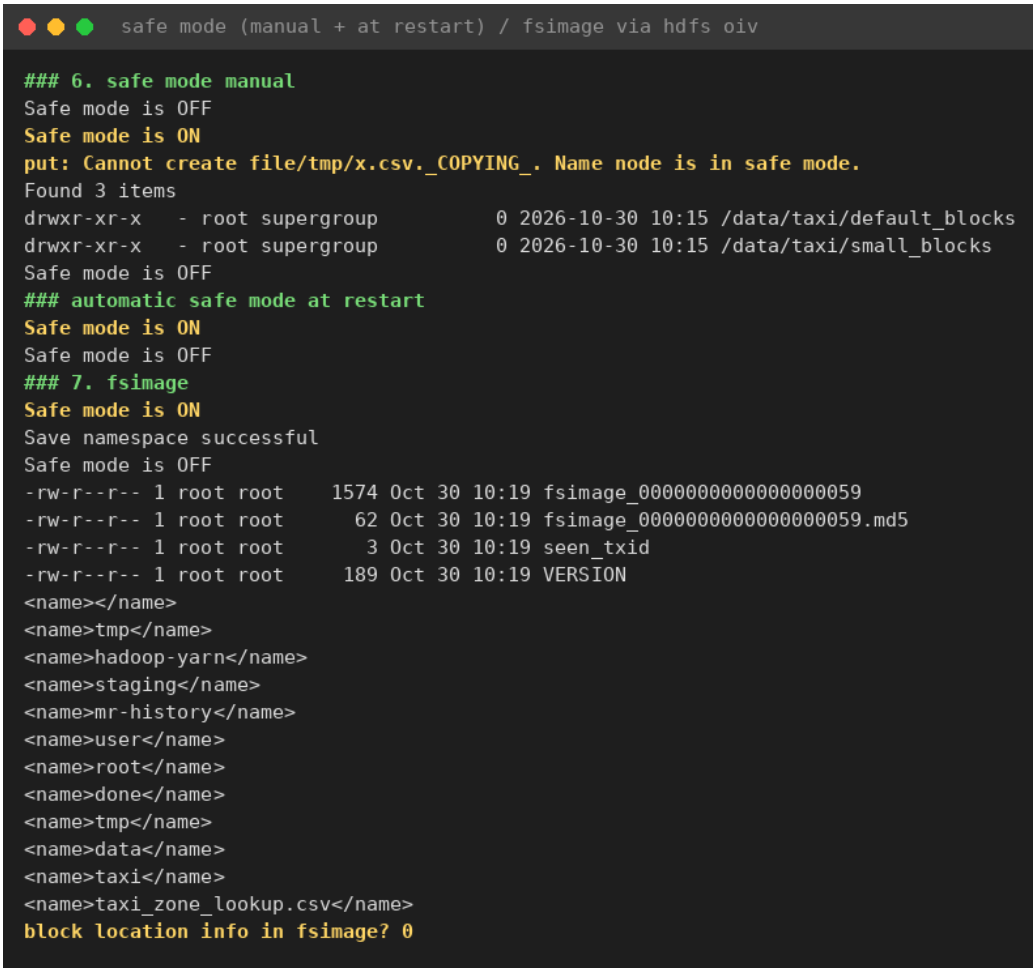


Hình 14. Trạng thái cluster sau khi datanode3 quay lại: không còn block under-replicated hay over-replicated.

2.6 Safe mode và fsimage

Khi bật safe mode thủ công, lệnh ghi bị từ chối với thông báo `put: Cannot create file/tmp/x.csv._COPYING_. Name node is in safe mode.`, còn `hdfs dfs -ls /data/taxi` vẫn trả về 3 mục (Hình 15). Sau `docker compose restart namenode`, `-safemode get` báo `Safe mode is ON` trong vài giây đầu rồi chuyển sang OFF khi tỉ lệ block được báo cáo đạt ngưỡng 99.9 % (`threshold-pct`) cộng 5 s extension.

`saveNamespace` tạo `fsimage_0000000000000000059` dung lượng 1,574 byte (kèm `.md5` 62 byte, `seen_txid` 3 byte, `VERSION` 189 byte). Bản XML do `hdfs oiv` sinh ra có tên thư mục và tệp (`tmp`, `hadoop-yarn`, `user`, `root`, `data`, `taxi`, `taxi_zone_lookup.csv`, …) cùng block ID, nhưng tìm thông tin DataNode (cổng 9866) thì được 0 dòng: fsimage không chứa vị trí block.



Hình 15. Safe mode thủ công (ghi bị từ chối, đọc vẫn hoạt động), safe mode tự động khi khởi động lại và nội dung fsimage.

3 Trả lời câu hỏi checkpoint

3.1 Tệp 300 MB với cấu hình mặc định chiếm bao nhiêu block, replica và dung lượng?

Số block là $\lceil 300/128 \rceil = 3$: hai block 128 MB và một block cuối 44 MB. Với hệ số 3 có $3 \times 3 = 9$ replica và khoảng $3 \times 300 = 900$ MB dung lượng thô, cộng thêm tệp `.meta` khoảng 0.78 % (Bảng 4). Block cuối không chiếm trọn 128 MB (Mục 3.2), và phép nhân ba dung lượng khớp với số đo ở Bảng 5.

3.2 Vì sao block cuối của tệp taxi nhỏ hơn 16 MB, và có tốn 16 MB đĩa không?

Vì tệp kết thúc ở đó: $49{,}961{,}641 - 2 \times 16{,}777{,}216 = 16{,}407{,}209$ byte. Tệp `blk_1073741830` trên datanode1 đúng bằng kích thước này (Bảng 4), nên không có phần đĩa nào bị bỏ phí, khác với block của hệ thống tệp cục bộ (ví dụ 4 KB) vốn được cấp phát trọn. Tệp 12,331 byte được gán block size 128 MB cũng chỉ chiếm 12,331 byte trên mỗi DataNode.

3.3 Vị trí block được lưu ở đâu, và khi NameNode khởi động lại thì sao?

Ánh xạ block sang DataNode chỉ nằm trong RAM của NameNode: fsimage có tên tệp và block ID nhưng không có thông tin DataNode nào (Mục 2.6, Hình 15). Khi khởi động lại, NameNode nạp namespace từ fsimage và edit log rồi dựng lại block map từ block report, nên phải ở safe mode cho tới khi nhận đủ báo cáo. Thiết kế này hợp lý vì chỉ DataNode mới biết chắc block nào đang nằm trên đĩa của nó; trong lúc NameNode tắt, đĩa có thể hỏng hoặc node bị thay, nên danh sách vị trí lưu sẵn sẽ nhanh chóng sai [1], [2]. Hiện tượng thiếu replica tạm thời ngay sau khi ghi (Mục 2.3) cũng xuất phát từ đây.

3.4 Vì sao tệp hệ số 2 được phục hồi còn tệp hệ số 3 thì không?

Muốn tái nhân bản phải có một DataNode còn sống chưa giữ replica của block, vì HDFS không đặt hai replica của cùng block trên một node [3]. Mỗi block của tệp hệ số 2 ban đầu nằm trên datanode1 và datanode3, nên datanode2 còn trống để nhận bản sao mới (Bảng 7, Hình 12). Block của hai tệp hệ số 3 thì đã có mặt trên cả hai node còn sống (Bảng 8), không còn chỗ cho bản thứ ba, nên phải chờ datanode3 quay lại (Hình 14). Số replica tối đa đạt được là $\min(\text{hệ số đích}, \text{số DataNode sống})$.

3.5 `fsck` báo gì khi có block under-replicated, và vì sao đó chưa phải lỗi?

`fsck /data` vẫn báo `Status: HEALTHY` khi có block under-replicated (Hình 13). HEALTHY nghĩa là mọi block còn ít nhất `dfs.namenode.replication.min` (= 1) replica sống, nên dữ liệu vẫn đọc được, và md5 trùng khớp ở Mục 2.5 xác nhận điều này. Under-replication chỉ làm

giảm độ dư thừa: hỏng thêm một node thì các block này còn một bản. Lỗi thật là MISSING hoặc CORRUPT, khi block không còn replica hợp lệ nào; `/tmp/one_replica.csv` sẽ rơi vào trường hợp này nếu replica duy nhất của nó nằm trên datanode3.

3.6 Nếu giữ `recheck-interval` mặc định 5 phút thì sao, và vì sao timeout dài lại tốt?

Theo công thức ở Mục 2.5, node sẽ bị tuyên bố chết sau $2 \times 300 + 10 \times 3 = 630$ s, tức 10.5 phút, thay vì 90 s như Bảng 6. Trong môi trường thật, timeout dài tránh được *replication storm* (bão tái nhân bản): một node khởi động lại để vá lỗi hoặc mất mạng một phút cũng có thể kéo theo việc sao chép hàng terabyte block, và khi node quay lại thì các bản sao đó thành thừa, trong khi băng thông và I/O đĩa bị tiêu tốn đúng lúc cluster đang không ổn định. Hai ngưỡng phục vụ hai việc khác nhau: ngưỡng stale ngắn (mặc định 30 s, trong lab 15 s) giúp client tránh node bất thường gần như ngay, còn ngưỡng dead dài chỉ quyết định lúc bắt đầu tái nhân bản, việc tốn kém nhất.

3.7 Vì sao NameNode khởi động trong safe mode và phải chặn ghi?

Lúc khởi động, NameNode biết namespace nhưng chưa biết block nằm ở đâu (Mục 3.3), nên khi chưa đủ 99.9 % block được báo cáo, nó không phân biệt được block thật sự mất với block chỉ chưa kịp báo. Nếu cho ghi, xóa hay tái nhân bản lúc này, NameNode có thể sao chép hàng loạt block chưa được báo cáo hoặc xóa nhầm replica tưởng là thừa. Safe mode giữ hệ thống chỉ đọc cho tới khi block map đáng tin (Mục 2.6).

3.8 Vì sao 1 triệu tệp 1 KB là vấn đề, còn 1 tệp 1 KB thì không?

Mỗi tệp, thư mục và block là một đối tượng trong heap NameNode, khoảng 150 byte mỗi đối tượng [4]. Một triệu tệp nhỏ tạo ít nhất 2 triệu đối tượng (một inode và một block mỗi tệp), xấp xỉ 300 MB heap cho khoảng 1 GB dữ liệu. Khi ghi còn có 1 triệu lời gọi RPC tới NameNode, và khi xử lý bằng MapReduce thì có 1 triệu map task vì mỗi tệp tạo ít nhất một split (*small files problem*). Trong thực nghiệm, tệp 12,331 byte vẫn chiếm riêng một block ID và một replica trên mỗi DataNode (Bảng 4). Cách xử lý thường dùng là gộp tệp nhỏ thành tệp lớn hoặc dùng định dạng chứa như Parquet/ORC, SequenceFile, HAR.

4 Kết luận

Bài lab đã hoàn thành các bước 1 đến 7 của đề. Block size chỉ là giới hạn trên và dung lượng thô tỉ lệ với hệ số nhân bản. Khi dừng datanode3, node bị tuyên bố chết sau 90 s như công thức, tệp hệ số 2 tự tái nhân bản sang datanode2, còn hai tệp hệ số 3 under-replicated nhưng vẫn HEALTHY và đọc được. fsimage không chứa vị trí block, đó là lý do NameNode cần safe mode khi khởi động.

Tài liệu tham khảo

[1] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The Google File System," in *Proc. 19th ACM Symposium on Operating Systems Principles (SOSP)*, 2003, pp. 29-43.

[2] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop Distributed File System," in *Proc. IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1-10.

[3] Apache Software Foundation, "HDFS Architecture," Apache Hadoop 3.4 Documentation. https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html

[4] T. White, *Hadoop: The Definitive Guide*, 4th ed. Sebastopol, CA, USA: O'Reilly Media, 2015.

[5] Apache Software Foundation, "hdfs-default.xml," Apache Hadoop 3.4 Documentation. https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/hdfs-default.xml

Phụ lục: Tệp nộp kèm và trích đoạn lệnh

Các log nộp kèm được liệt kê ở Bảng 9; ảnh chụp nằm trong `img/`.

Bảng 9. Danh mục tệp log nộp kèm báo cáo.

Tệp	Nội dung
`01_shell_blocks.txt`	`dfsadmin -report`, lệnh `hdfs dfs -stat`, `fsck` block 128 MB/16 MB, tệp block
`02_setrep.txt`	`fsck` tệp 16 MB khi đã ổn định, `setrep -w 2`, `du -h`, ghi tệp hệ số 1
`03_` đến `07_*.txt`	`fsck` trước/sau khi dừng datanode3, dòng thời gian, md5, datanode3 quay lại
`08_safemode_fsimage.txt`	Safe mode thủ công và tự động, `saveNamespace`, phân tích fsimage

In tên, block size, hệ số nhân bản và kích thước của `taxi_zone_lookup.csv`:

```
hdfs dfs -stat "name=%n blocksize=%o repl=%r size=%b"
 ↪  /data/taxi/taxi_zone_lookup.csv
# name=taxi_zone_lookup.csv blocksize=134217728 repl=3 size=12331
```

Ghi tệp với hệ số 1 trong một lệnh, không dùng `setrep`:

```
hdfs dfs -D dfs.replication=1 -put
 ↪  /datasets/data/taxi/taxi_zone_lookup.csv /tmp/one_replica.csv
# one_replica.csv repl=1   (replica duy nhất trên 172.29.0.5 = datanode1)
```

Các lệnh chính của thí nghiệm block và sự cố DataNode:

```
hdfs dfs -D dfs.blocksize=16m -put
 ↪  /datasets/data/taxi/yellow_tripdata_2024-01.parquet \
    /data/taxi/small_blocks/
hdfs fsck /data/taxi/small_blocks -files -blocks -locations
hdfs dfs -setrep -w 2
 ↪  /data/taxi/small_blocks/yellow_tripdata_2024-01.parquet
docker compose stop datanode3          # chạy trên máy host
while true; do hdfs dfsadmin -report | grep -E "Live|Dead|Under"; sleep 5;
 ↪  done
hdfs dfs -get /data/taxi/default_blocks/yellow_tripdata_2024-01.parquet
 ↪  /tmp/check.parquet
md5sum /tmp/check.parquet
 ↪  /datasets/data/taxi/yellow_tripdata_2024-01.parquet
```